CuPath: Continuous Memory-Request Transformation for Wafer-Scale GPUs

HPCA 2027 Submission **#NaN**
Confidential Draft
Do NOT Distribute!!

***Abstract*—Wafer-scale GPUs expose distributed compute and HBM as one logical device, yet a cache-line miss may traverse requester L2, RDMA endpoints, the wafer network, owner L2, and HBM. Existing designs optimize each component using only locally visible information. We show that optimization opportunities change during a request lifetime: real accesses expose a predictive relation, L2 state eliminates already resident or pending work, requester RDMA exposes cross-L1 and destination aggregation, and returned data exposes reuse. We propose *CuPath*, which connects these decisions through a common bounded real-demand predictor design and one typed Cuckoo Filter per L2 slice. CuPath first filters a directly adjacent local candidate and, when useful, exposes it with the triggering miss through an aligned 128-B controller-level HBM burst, then permits a remote candidate only to piggyback an existing owner/page batch, and finally retains useful returned data in the existing requester L2 while feeding outcomes back to prediction metadata. Exact tags, MSHRs, and transaction tables remain authoritative; unavailable speculative opportunities are dropped immediately. CuPath adds no global scheduler, new cache level, or cacheline-data store. We evaluate both end-to-end performance and the cache, DRAM, RDMA, packet, and wafer-traversal work removed at each boundary.**

I. Introduction

The performance demand of GPU workloads continues to grow, while the compute, memory capacity, and bandwidth of a single die scale more slowly. Multi-chip GPUs increase these resources within a package [**?**], [**?**]. Wafer-scale GPUs go further: they connect many compute chiplets and distributed HBM stacks with a high-bandwidth, low-latency on-wafer network and expose the system as one logical GPU [**?**], [**?**], [**?**]. This organization greatly increases aggregate resources, but it does not make every memory access local.

Each memory page still belongs to the HBM of one GPU processing module (GPM). When a requester accesses a local page, an L1 miss proceeds through its L2 and HBM. When it accesses a remote page, the same miss may pass through requester L2, requester RDMA, the wafer network, owner RDMA, owner L2, and owner HBM; the response then travels back across the wafer. Thus, one remote cacheline can consume queues, cache lookups, packets, links, and memory bandwidth at several GPMs. Prior multi-GPU studies show that this communication cost and the underlying sharing behavior vary widely across applications [**?**], [**?**].

Modern memory systems optimize each component separately. Caches merge local misses and exploit temporal locality, DRAM controllers schedule requests for bank and row locality, and RDMA engines move data between memory domains. These optimizations are important, but each component can act only on the information available at its own input. As a result, an unnecessary request that escapes an early component continues to consume work at every later component. For example, two L1 caches may send the same remote line because their private MSHRs cannot see each other. The owner L2 can eventually merge the two misses, but only after both have consumed requester-RDMA and network resources. Similarly, returning every remote line only to requester L1 causes a later reuse to repeat the entire wafer traversal, while blindly placing every return in requester L2 can pollute useful local data [**?**], [**?**].

The key observation of this work is simple: a request reveals different useful information as it moves through the memory path. Real L2 misses reveal whether adjacent lines follow a predictable pattern. L2 then reveals whether a predicted line is already cached or in flight. Requester RDMA sees requests from many private L1s at once, exposing duplicate lines and requests headed to the same owner and page. Finally, a returned line reveals whether the data served multiple waiters or is likely to be reused. No single component sees all of this information, but together these observations can prevent unnecessary work before it reaches the next expensive stage.

Our characterization confirms that these opportunities are both significant and complementary. Repeated real demands produce predictable relations in all representative workloads, but many predicted candidates are already resident or pending. Across remote traffic, 26.5% of sampled reads duplicate an in-flight cacheline; after removing those duplicates, 22.2% of the remaining reads have a nearby same-page partner. Remote reuse is less common but concentrated: only 17.6% of unique remote lines recur, yet later touches account for 33.6% of remote reads. These results motivate optimizing the request at several points along its path rather than adding one more isolated cache, prefetcher, or network scheduler.

We propose *CuPath*, which uses a lightweight demand predictor and one typed Cuckoo Filter per L2 slice to identify and optimize memory requests as they move through the memory path [**?**]. The predictor proposes at most one adjacent candidate from real demand history. The Filter records compact pattern, residency, in-flight, and reuse information, allowing later stages to quickly reject work that is already present or unlikely to help. The Filter is a control structure rather than a data cache: exact tags, MSHRs, and RDMA

Fig. 1. CuPath continuously filters a predicted request, aggregates it only with compatible in-flight work, and retains returned value in the existing requester L2.

transaction tables remain authoritative.

CuPath applies this information in three places. First, at the local L2, it removes resident or pending candidates and, when the remaining sibling is compatible with the demand, exposes both 64-B lines through one aligned 128-B controller-level HBM burst. Second, at requester RDMA, it merges identical remote requests and groups different lines going to the same owner and page; a predicted remote line is allowed only when it can occupy a free position in an existing batch. Third, after the response returns, CuPath uses real reuse evidence to retain useful remote data in the existing requester L2, avoiding a later wafer traversal. Outcomes feed back to the predictor so that useful relations remain active and unused ones are suppressed.

CuPath is deliberately work-conserving. It never delays a demand while waiting for a partner, never creates a remote packet for a predicted line, and immediately falls back to the original path when metadata or downstream resources are unavailable. It adds no global scheduler, new cache level, or cacheline-data store. Figure **??** summarizes how a request is filtered, combined with useful in-flight work, and selectively retained as it moves through the system.

Across fourteen GPU workloads, CuPath improves every workload and achieves a 1.534× geometric-mean speedup. More importantly, the speedup follows from work removed along the path: CuPath reduces L2 tag lookups by 22.6%, on-wafer packets by 48.2%, and repeated wafer traversals by 30.8%.

This paper makes the following contributions:

- We identify and quantify how predictive locality, cache state, remote aggregation, and reuse become visible at different points in a wafer-scale memory-request lifetime.
- We propose CuPath, an end-to-end memory-path design that couples a bounded real-demand predictor and per-slice typed Cuckoo Filter with the existing HBM controller, RDMA engine, and requester L2.
- We implement and evaluate CuPath without adding a global scheduler, cache level, or cacheline-data store, achieving a 1.534× geometric-mean speedup while substantially reducing cache and network work.

II. Background

A. Wafer-Scale GPU Memory Organization

Wafer-scale integration connects many GPU Processing Modules (GPMs) through a high-bandwidth on-wafer network [**?**], [**?**], [**?**]. Each GPM contains compute units, private L1 caches, a shared L2, an RDMA endpoint, and a local HBM stack. Its L2 is divided into address-interleaved *slices*, each serving a subset of addresses [**?**], [**?**]. Physical pages are distributed across GPMs. We call the GPM issuing an access the *requester* and the GPM holding the page the *owner*. An owner-local access uses the local cache and HBM hierarchy; a remote access additionally crosses requester RDMA, the network, and owner-side resources.

B. Baseline Local and Remote Memory Paths

A wavefront coalescer converts lane accesses into cache-line requests. Each request first searches L1, whose miss-status holding registers (MSHRs) merge concurrent same-line misses. The page owner selects the local or remote path.

Local access. An L1 miss enters the address-selected local L2 slice. Both hits and misses pay its tag lookup latency. A hit returns the line; a miss allocates an L2 MSHR and reads local DRAM. The response fills L2 and completes all requests merged into that MSHR. This coalescing combines only the same line while its miss is in flight: adjacent 64-B lines remain separate transactions even when they map to the same DRAM row.

Remote access. If another GPM owns the page, the requester sends the L1 miss through its RDMA endpoint and the mesh. The owner endpoint injects the request into the owner-L2 slice; an L2 miss reads owner HBM. The response returns through owner RDMA, the network, and requester RDMA before filling requester L1. Requests from different L1s can therefore duplicate the same remote line, yet meet at owner L2 only after consuming endpoint and network resources. Because the baseline retains no shared requester-L2 copy, reuse after L1 eviction repeats the remote path.

C. Cache Filtering and Remote Aggregation

Requester-L2 caching can shorten a repeated remote access, but admitting every remote return wastes capacity on one-touch data [**?**], [**?**]. Admission should therefore require reuse evidence. Most first-touch remote lines are absent, so probing requester L2 unconditionally wastes latency and ports. A Cuckoo Filter supports compact approximate membership tests [**?**]. With updates on fills and evictions, a negative result safely skips the lookup; a positive result still requires exact tags. Filtering therefore removes work without affecting correctness [**?**].

RDMA endpoints also have finite processing width, queues, and outstanding capacity [**?**], [**?**]. Same-line requests visible concurrently at one requester can share a remote transaction when an exact table retains every waiter. Different lines going to the same owner and page can instead share a packet. Thus, *deduplication* reduces remote transactions, whereas *batching* reduces packet overhead.

D. DRAM Organization and Scheduling

DRAM is divided into channels, banks, rows, and columns. A bank exposes one active row through its row buffer. Accessing that row is a row hit; accessing a different row requires precharge and activation before transferring data. Serving several columns from one open row is therefore cheaper than repeatedly switching rows [**?**]. HBM exposes wide channels, many banks and pseudo-channels, and a burst-oriented controller interface. Prior GPU/HBM work observes that accesses within one aligned 128-B region can be coalesced into one controller request even though the HBM device services that request using finer-grained transfers [**?**]. Empirical HBM characterization further shows that address mapping, locality, and the request stream presented to the controller strongly affect attainable bandwidth [**?**]. More generally, MAC demonstrates that coalescing requests before 3D-stacked memory can improve both interface efficiency and performance [**?**].

This abstraction also has direct implementation precedent. AMD's RAMA HBM controller normalizes traffic into 64-B or 128-B burst fragments before the HBM subsystem [**?**]. We therefore configure the modeled HBM controller with an aligned 128-B burst fragment. This is a *controller-level transfer granularity*, not a new native HBM device command: the real controller may realize the fragment with its existing BL4 transfers, bank mapping, and row-buffer operation [**?**], [**?**]. Baseline and every CuPath configuration use the same HBM organization. A conventional 64-B cache miss consumes only its requested half of the modeled fragment. M1 predicts whether the adjacent 64-B half will be useful and, only then, exposes both halves to the cache hierarchy through the same aligned burst.

III. Observations: Evolving Information Visibility

We use six observations to follow the information available over a request lifetime. The first establishes the end-to-end cost; the remaining observations identify relationships that become actionable only at a particular requester, network, or owner-memory boundary.

O1: Cost of a Full Memory Path. We decompose complete local-DRAM and remote paths using only the MSHR leader that issues each physical request. Local links are charged to their receiving cache or DRAM, while endpoint processing and both wafer traversals are charged to Remote Communication; each path is then normalized to 100%. As Fig. **??** shows, the dominant local component varies by workload, while communication often dominates a remote path even when it ends at the owner L2. The critical path therefore spans several visibility boundaries rather than one uniformly dominant component.

O2: Real Demands Expose Predictive Relations. We feed only real L2 read demands to the bounded stride detector and count a candidate only after a repeated relation has been observed. Figure **??** shows that all seven representative workloads expose qualifying relations, with candidates covering 1.8%–30.7% of real demands. The opportunity is therefore neither limited to contiguous streams nor universal enough to issue blindly; prediction must remain selective and retain an immediate fallback.

Fig. 2. Component breakdown of local DRAM paths (L) and complete remote paths (R). We exclude MSHR followers and normalize each qualifying path class to 100%. "–" means that the bounded O1 observation window contains no qualifying remote leader; it does not imply that the complete application never accesses remote memory. Local transport is charged to its receiving cache, while RDMA and wafer traversal are reported as Remote Communication.

Fig. 3. O2: repeated real-stride evidence and generated 64-B candidates. Speculative accesses do not train the detector.

O3: Membership State Separates Useful from Redundant Work. An ungated predictor frequently proposes lines that

should not consume the memory path. Across the representative screen, typed membership checks remove 34.7% of issued candidates while retaining 96.7% of useful candidates (Fig. **??**); useful-per-issued accuracy rises from 24.0% to 35.5%. Residency and in-flight state are therefore most valuable when coupled to candidate admission, before unnecessary DRAM work is created.

Fig. 4. O3: work removed and useful work retained by typed Filter gating, normalized to ungated prediction.

O4: Remote Misses Amplify Wafer-Wide Work. For each logical remote request, we record endpoint service, forward and return traffic, Manhattan distance, and completion latency. Across 713,581 sampled requests from ten workloads, one logical byte produces 5.50 network byte-hops and mean remote latency ranges from 464 ns to 11.86 μs (Fig. **??**). Eliminating one remote transaction therefore removes work from multiple components in both path directions.

O5: Network Visibility Exposes Two Aggregation Scopes. We separate exact same-line in-flight redundancy from distinct lines that share a requester, owner, and 4-KB page. Exact predecessors cover 26.5% of sampled remote reads; after removing them, 22.2% of the remaining reads find a same-page partner within 16 cycles (Fig. **??**). MM is dominated by exact redundancy whereas MT exposes spatial packet grouping, motivating both requester-side coalescing and owner/page packet aggregation. The 16-cycle interval is used only to characterize proximity. M2 joins a candidate only to a batch that already exists and never waits for a future partner.

O6: Completed Transactions Reveal Selective Remote Reuse. We count completed independent remote transactions and sample free capacity in each requester-L2 slice. Only 17.6% of unique remote lines recur, but later touches constitute 33.6% of remote reads; mean free L2 capacity is 62.4% and

Fig. 5. O4: network byte-hops per logical remote byte and mean end-to-end remote-transaction latency. "–" denotes no remote request in the O4 window.

Fig. 6. O5: exact in-flight coalescing and same-page, different-line packet aggregation opportunities. The spatial bar uses a 16-cycle requester RDMA window; "–" denotes no remote read.

ranges from 9.8% to 93.8% across workloads (Fig. **??**). CuPath therefore caches remote lines at requester L2 selectively: real recurrence or multiple waiters justify normal remote-clean admission, while a first-touch speculative response can use only an already-invalid victim and cannot displace a valid line.

Fig. 7. O6: recurring remote lines, reads beyond first touch, and mean free requester-L2 capacity. "–" denotes no remote read; L2 capacity is still reported for those workloads.

Synthesis: Visibility Evolves Over the Request Lifetime. O1–O6 are not six views of one opportunity. Real demands expose predictive relations, L2 exposes residency and in-flight state, requester RDMA exposes cross-L1 concurrency and common destinations, and completed remote transactions expose temporal recurrence. None is fully available at request generation. This motivates a lifecycle that acts when each relationship first becomes actionable, so an upstream decision can eliminate work before it reaches downstream components.

IV. Design

CuPath follows a memory request across the local and remote paths. At each boundary, it uses the information that has become visible there to eliminate unnecessary work or to attach useful work to an operation that must already occur. The Filter provides a compact control plane for these decisions; it never stores data and never replaces the exact cache, MSHR, or RDMA state. Figure **??** summarizes this request lifecycle.

A. Shared Control Plane

Real-demand candidate generation. CuPath uses a small, direct-mapped predictor near the memory-path front end. Each entry records the source, last real cacheline, last real stride, minimal repeated-stride evidence, and a generation. Repeating a stride with real demands establishes a pattern and may propose at most one 64-B candidate. Speculative accesses never train the predictor. A generation token prevents feedback for a replaced pattern from changing new state. The four L2 slices in a GPM share one bounded local predictor, while requester RDMA has one bounded remote instance per GPM. All instances use the same state format and prediction rule; useful or unused response feedback returns through the generation token to the instance that proposed the candidate.

One typed Filter per L2 slice. Each slice has one typed Cuckoo Filter [**?**]; RDMA addresses the Filter beside the requester-L2 slice and does not instantiate another one. Five logical key types share its bucket array:

- PATTERN says that real demands established a predictor relation;
- RESIDENT says that a line may exist in the current L2 slice;
- PENDING says that a remote line request may already be active;
- SEEN records recurrence evidence produced by real remote activity; and
- LOCAL-PENDING says that a local line already has an ordinary demand MSHR or is attached to a paired read.

RESIDENT, PENDING, and LOCAL-PENDING have priority over the lower-priority PATTERN and SEEN classes. The type participates in the hashes and stored fingerprint; a small reference count supports duplicate signatures. With four slices per GPM and 48 GPMs, CuPath has 192 physical Filters—not four independent metadata arrays per slice.

The Filter is never authoritative for data or full identity. A possible match is confirmed by an exact tag, MSHR, or RDMA line entry. If critical metadata is unreliable, a demand fails open to its ordinary path. If a speculative lookup, update, or downstream resource is unavailable, CuPath drops only the candidate. Lookup and update width and latency are modeled; there is no implicit zero-cycle oracle.

B. M1: Predictive HBM Burst Utilization

An L1 miss normally continues as an isolated 64-B request, even when its adjacent cacheline is already waiting nearby or will be requested shortly. Neither L1 nor DRAM alone can safely exploit both cases: L1 does not know the shared L2 state or physical DRAM mapping, while DRAM sees the relationship only after the requests have been independently scheduled. M1 carries this relationship across the boundary. It forms a logical pair $\langle D, S \rangle$ between a mandatory demand D and at most one adjacent sibling S, then exposes the surviving pair as one aligned 128-B controller-level HBM burst. L1 and L2 retain their 64-B cachelines; the wider request changes neither cache capacity nor the HBM device interface. Figure **??** shows the workflow.

Expose an adjacent opportunity. M1 learns about a sibling in one of two ways. First, when a real CU memory instruction emits both adjacent lines, the L1 coalescer marks only the later request with a one-bit hint. The hint carries no address and cannot create a request; it merely indicates that an adjacent real miss may already be pending. Second, repeated real-demand strides may predict that the adjacent line will be needed soon.

Speculative requests never train this predictor, and each stream can have at most one unverified sibling, preventing a chain of speculative accesses.

Let Filter-visible state choose the action. On a hinted request, the slice queries LOCAL-PENDING. A reliable negative eliminates the exact sibling-MSHR lookup. A possible match is confirmed by the MSHR; if the adjacent real demand is ready but unissued, M1 pairs the two existing misses immediately. This preferred path adds no cacheline, MSHR, or DRAM access.

A predicted sibling is admitted only after PATTERN, RESIDENT, and LOCAL-PENDING classify it as an established relation that is neither cached nor already in flight. Approximate positives are confirmed by the exact predictor generation, tag, or MSHR state. The sibling must also remain in the same page, L2 slice, and memory controller as D, and the ordinary L2 and DRAM resources for both reads must be available. Only then does M1 allocate one normal L2 MSHR and issue the sibling early. Thus the predictor proposes an address, but the Filter decides whether that proposal represents useful new work.

All Filter operations overlap the normal processing of D. If the metadata or resources are unavailable when D is ready, M1 drops the opportunity and issues D alone. It never holds a demand to wait for a peer. Consequently, every request reaches one of three work-conserving outcomes:

1) pair two already-existing real misses;
2) attach one screened early sibling to the mandatory miss; or
3) preserve the original singleton request.

Use the existing HBM burst fragment. If a sibling survives the checks above, L2 aligns the request to the enclosing 128-B region and requests both 64-B halves through one controller-level burst. This organization follows prior GPU/HBM studies of aligned 128-B requests and commercial HBM-controller support for 64-B or 128-B burst fragments [**?**], [**?**]. M1 does not add a 128-B JEDEC command or widen the physical HBM channel. It decides whether data already transferable by the configured HBM burst should include the adjacent cacheline. The controller remains responsible for realizing the burst with its ordinary bank, row, and BL4 operations. If the predictor is not confident or the 128-B region crosses an incompatible mapping boundary, D immediately uses the ordinary 64-B request path.

Use demand feedback to limit new work. An early sibling fills the existing L2, not a new cache. A demand that merges before the fill marks it *late*; a later L2 hit marks it *timely*; eviction before use marks it *unused*. Generation-tagged feedback updates only the stream that produced the sibling. Timely or late use may release one subsequent proposal, whereas unused data suppresses further speculation until the stale pattern retires. Pairs formed from two real misses require no such feedback because both members are useful by construction.

M1 is therefore one Filter-guided transformation rather than an unconditional prefetcher. Real requests expose adjacency, the Cuckoo Filter removes redundant or unsafe work, and the surviving relation determines whether the configured HBM controller burst carries one useful cacheline or two. At every failed decision, the request immediately returns to the unchanged 64-B path.

Figure/M1.png

Fig. 8. M1 uses Filter-visible state to admit an adjacent sibling, expose both 64-B halves through one aligned controller-level HBM burst, or fall back to the ordinary 64-B path.

C. M2: Prefetch-Aware Remote Aggregation

Remote accesses reveal relationships across private L1s. Requester RDMA therefore applies the same predictor format while retaining exact same-line deduplication: requests with identical PID, owner, and cacheline share one line entry and exact waiter list. Unique real leaders with the same owner, PID, and 4-KB page can share a bitmap packet. Outstanding entries, batches, owner expansion, and response fanout remain bounded by the modeled RDMA capacity and width.

M2 does not allow prediction to create independent remote traffic. A candidate must map to the same owner and page as its triggering demand, pass PATTERN/RESIDENT/PENDING and exact line-table checks, and find a free bitmap position in an *existing* batch. Only then does M2 insert PENDING and piggyback the line. No matching batch, a full batch, or a busy metadata port drops the candidate immediately. The batch is never delayed for it, and a candidate can never create a packet.

If a real demand later names the speculative line, it joins the exact waiter list rather than injecting another request. One owner access and one returned line then satisfy all waiters. Thus the Filter couples prediction to the aggregation opportunity that becomes visible at RDMA, instead of merely throttling a standalone remote prefetcher.

D. M3: Filter-Coupled Remote Reuse

After exact in-flight merging finds no existing transaction, M3 uses RESIDENT and SEEN as a requester-L2 probe gate. A reliable negative for both classes skips an otherwise guaranteed-miss requester-L2 tag lookup and lets the request proceed immediately to RDMA aggregation. A positive or unreliable result performs the exact requester-L2 lookup: a hit completes locally, while a miss continues to the owner. Disabling the Filter also takes this fail-open, always-probe path. Thus the approximate metadata never supplies data or

Fig. 9. M2 admits a remote candidate only as a free line in an existing owner/page batch; exact line state deduplicates later demand.

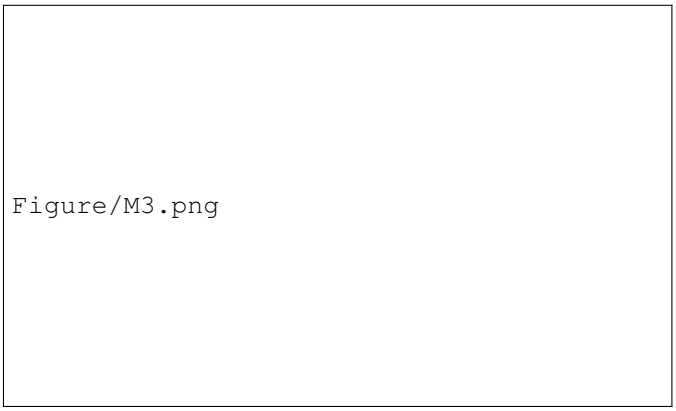

Fig. 10. M3 first gates a requester-L2 probe with RESIDENT/ SEEN; after a remote response, real-use evidence controls best-effort retention in the same existing L2 and lifecycle feedback.

suppresses a possible hit; it only removes known-miss tag work before the remote traversal.

The returned line then reveals whether speculation became useful and whether a second wafer traversal is likely. M3 uses that information in the *existing requester L2*; CuPath adds no L1.5 cache or cacheline-data store. Response arrival removes PENDING and first serves the exact waiter list at bounded fanout width.

A line with multiple real waiters or prior real SEEN evidence is eligible for a best-effort clean fill under the requester L2's remote-clean replacement policy. A predicted line that is consumed by a real waiter is also eligible because prediction and demand have exposed recurrence before the response arrives. A first-touch speculative line without such evidence is more restricted: it may occupy an invalid way, but cannot replace any valid local or remote line. All fills also reject dirty, locked, active-MSHR, or stale-generation victims. Successful fill establishes RESIDENT.

A later real requester-L2 hit avoids RDMA, the wafer network, and owner memory, marks the retained line useful, and leaves its real-demand-established PATTERN eligible. A speculative line evicted before use removes that PATTERN; candidates that retire earlier in the RDMA lifecycle also penalize their predictor generation. Only real remote activity can establish SEEN; a speculative access cannot recursively justify more speculation. Remote writes advance exact line epochs and invalidate stale recurrence metadata.

Across the three stages, the policy is the same: propose from real evidence, filter with newly visible state, combine with useful in-flight work when possible, and otherwise fall back immediately. This continuous control flow is the connection among M1–M3 and requires no global scheduler.

V. EVALUATION METHODOLOGY

A. Simulation Infrastructure

Simulation. We extend MGPUSim [**?**] with the distributed memory, packet-switched mesh, bounded RDMA endpoints, and cache-path instrumentation required to model a wafer-scale GPU. The modeled system contains 48 GPMs in a 7×7 mesh whose remaining tile hosts the central I/O and control components. Each network link provides 768 GB/s of bandwidth and incurs 32 cycles of switch latency, consistent with prior multi-chip and wafer-scale studies [**?**], [**?**]. Each GPM represents one quarter of an AMD MI100-class GPU; four GPMs collectively provide the compute, cache, memory-capacity, and memory-bandwidth resources of one MI100 [**?**]. This scaling keeps cycle-level simulation of all 48 GPMs tractable while preserving the aggregate resource ratios.

Cache and memory hierarchy. Each GPM contains 32 CUs organized as eight shader arrays. Every CU has a private 16-KB, four-way L1 vector cache with a 64-B line, 10-cycle bank access, 16 MSHRs, and at most 160 concurrent transactions. Each shader array also has a 16-KB scalar cache and a 32-KB instruction cache. The GPM-wide 4-MB L2 is address-interleaved across four 1-MB slices. Each slice is 16-way associative, accepts up to 16 requests per cycle, has 64 MSHRs, and charges a 10-cycle tag/directory lookup on both hits and misses. Four memory controllers connect the slices to 8 GB of local HBM with 1.23 TB/s aggregate bandwidth. Table **??** summarizes the modeled system and CuPath resources.

No additional cache level. The MCM-GPU design introduces a GPM-side, remote-only L1.5 cache between L1 and L2 and explores reallocating L2 capacity to that new level [**?**]. CuPath does not add an L1.5 cache, a victim cache, or any other cacheline-data store. Baseline and CuPath therefore use identical L1 and L2 capacities, associativities, ports, MSHRs, and lookup latencies. A reuse-qualified remote response is inserted as a clean line through the existing shared-L2 fill and replacement path; the Cuckoo Filters hold only metadata and never cache data. Consequently, CuPath's reported gains do not come from increasing or redistributing cache capacity.

Unlike an address-translation study, CuPath leaves the TLBs, page-table walkers, and 4-KB page mapping unchanged in every configuration. Translation determines the physical page owner, after which the measured data request enters the local or remote cache path described in Section **??**. Consequently, all reported differences arise below translation, from cache, DRAM, RDMA, and network work performed on the data request.

TABLE I
BASELINE WAFER-SCALE GPU AND CUPATH CONFIGURATION.

Module	Configuration
Wafer	48 GPMs, 1.0 GHz, 7×7 mesh
Compute	32 CUs/GPM, eight shader arrays/GPM
L1 vector	16 KB/CU, 4-way, 64 B, 10 cycles, 16 MSHRs
L1 scalar	16 KB/shader array, 4-way, 16 MSHRs
L1 instruction	32 KB/shader array, 4-way, 16 MSHRs
Shared L2	4 MB/GPM, four slices, 16-way, 64 MSHRs/slice, 16 req./cycle/slice, 10-cycle lookup
HBM	8 GB/GPM, four controllers, 1.23 TB/s, BL4, 128-B controller-level burst fragment
RDMA endpoint	8 transfers/cycle, 10-cycle processing, 64 outstanding/direction
Mesh link	768 GB/s, 32-cycle switch latency, 16-B flit
CuPath additions	
Cache capacity	No L1.5 or new data cache; baseline L1/L2 unchanged
Typed Filter	One shared array/L2 slice, 32K slots, four slots/bucket, 13-bit fingerprint
Filter ports	1-cycle lookup/update, 16 lookups and 16 updates/cycle/slice
Predictors	Two 256-entry tables/GPM in Complete (one shared local + one requester RDMA)
M1	Filter-guided use of an aligned 128-B HBM burst
M2	Up to eight lines/packet, 64 packet descriptors/requester
M3	No separate history; best-effort clean fill into existing L2

B. Ablation Configurations

We run five configurations from the same frozen simulator binary. *Baseline* disables every CuPath transformation. *M1* enables the complete local request-pairing path: reliable RESIDENT negatives bypass guaranteed-miss L2 tag lookups, and typed membership state admits at most one predicted sibling into the triggering miss's aligned 128-B HBM burst. *M2* enables exact remote same-line deduplication, work-conserving owner/page packets, and existing-batch-only remote candidate piggybacking; requester-L2 reuse is off. *M3* enables SEEN/RESIDENT-guided requester-L2 probe gating and reuse without remote batching or candidate generation. *Complete* enables M1–M3. Row-aware DRAM reordering remains disabled in all five configurations.

In the complete configuration, the stages operate on the surviving request population. A requester-L2 termination never enters requester RDMA, and a coalesced RDMA waiter does not create a network or owner-memory request. We therefore place counters at each boundary and report both eliminated work and end-to-end performance, rather than treating the three stage speedups as additive.

The CuPath entries in Table **??** list the finite resources used by Complete. Each 1-MB L2 slice has one 84-KiB typed Cuckoo Filter with twice as many slots as L2 blocks. Five logical key classes share that array and its modeled ports. Across all 192 slices, the Filter arrays total 15.75 MiB, or 8.20% of baseline L2 data capacity. Possible matches are verified by exact cache tags, MSHRs, or RDMA transaction state. The exact shadow used only to count false positives is simulator-only and excluded from the hardware cost. PATTERN/SEEN cannot consume the capacity reserved for RESIDENT/PENDING. Bitmap packets encode at most eight 64-B lines, and each requester holds at most 64 owner/page packet descriptors. M1 has no candidate waiting queue; M2 candidates can use only an already existing batch; and M3 adds no separate history table because SEEN shares the same Filter. Charging a conservative 64 B to every predictor entry—more than the encoded source, address, stride, evidence, generation, and Filter-slice owner require—adds 32 KiB/GPM for the two 256-entry tables in Complete, or 1.50 MiB across the wafer. Filter and predictor metadata together are thus at most 8.98% of baseline L2 data capacity under this conservative bound. At 32 nm, a matched CACTI array estimate gives 0.5900 mm^2 and 0.0691 nJ per two-bucket lookup for one Filter slice, respectively 1.29% of the area and 1.52% of the dynamic access energy of the modeled 1-MiB L2 slice. These are array-only lower bounds: the simulator additionally models aggregate Filter-port contention, but CACTI does not include hashing, fingerprint comparison, or intra-array bank conflicts.

C. Workloads and Execution

We evaluate CuPath with 14 benchmarks from Hetero-Mark [**?**], AMD APP SDK [**?**], SHOC [**?**], and DNNMark [**?**]. The suite has also been used by prior multi-GPU studies [**?**], [**?**], [**?**], [**?**] and spans streaming, adjacent, strided, iterative-reuse, and irregular gather/scatter behavior. Table **??** replaces translation-centric workload labels with the data-access behavior relevant to CuPath. It reports the workgroups actually admitted and retired in the formal run and the workload allocation footprint measured immediately after allocation from the simulator's 4-KB page counters.

We choose the largest hundreds-of-MB or GB-scale input that remains tractable in cycle-level simulation, following prior multi-GPU and wafer-scale studies [**?**], [**?**], [**?**]. Every footprint exceeds the capacity of a single GPM's private and shared caches, forcing requests to exercise local HBM and, under distributed page placement, remote owner paths.

Formal experiments do not use a positive workgroup cap. Every configuration executes the benchmark's original kernel launches and full mapped workgroup set. Warp-, branch-, and kernel-level sampling reduces cycle-level simulation cost without changing which workgroups are mapped; the same sampling controls are applied to Baseline and every CuPath configuration [**?**], [**?**]. The runner records every launch, partition, observed workgroup count, and a hash of the global workgroup set. We reject a comparison if these identities differ across configurations.

VI. RESULTS

A. End-to-End Performance

Figure **??** reports the frozen-binary formal ablation. All Baseline and Complete cells have finished, use identical sampling controls and admitted workgroups, and retire every admitted workgroup. Complete classifies all fourteen workloads as positive using our predeclared $1.005\times$ threshold and achieves a $1.534\times$ geometric-mean speedup. The All Local, Mixed, and Remote groups improve by $1.584\times$, $1.757\times$, and

TABLE II
BENCHMARKS, DATA-ACCESS BEHAVIOR, EXECUTION SCOPE, AND FOOTPRINT.

Benchmark	Data-access behavior	Execution	Footprint
AES	Block-structured reuse	Full	1,024 MiB
BT	Structured compare–exchange	Full	1,024 MiB
FWT	Butterfly, staged adjacency	Full	1,024 MiB
FFT	Butterfly and strided stages	Full	1,024 MiB
FIR	Adjacent streaming and reuse	Full	1,024 MiB
FWS	Dense matrix, repeated reuse	Full	1,024 MiB
I2C	Strided-to-contiguous gather	Full	1,029 MiB
KM	Iterative centroid reuse	Full	390 MiB
MM	Tiled dense reuse	Full	1,024 MiB
MT	Long-stride streaming	Full	1,013 MiB
PR	Irregular graph gather	Full	1,024 MiB
RELU	Contiguous streaming	Full	2,560 MiB
SC	Sliding-window stencil	Full	512 MiB
SPMV	Sparse indirect gather	Full	1,081 MiB

Fig. 11. Speedup over Baseline for the three standalone stages and Complete. Workloads are grouped by the dominant local/remote path.

1.187×, respectively. The gains are broad but not uniform: FIR, SC, and MM exceed 3×, whereas MT, SPMV, and PR improve by only 1.008×, 1.012×, and 1.033× because little reusable work survives their streaming or irregular paths.

M1, M2, and M3 independently reach 1.008×, 1.345×, and 1.233× geometric mean. M1 has nine positive, four neutral, and one negative workload; M2 has thirteen positive and one neutral workload; M3 has six positive and eight neutral workloads. Complete is not the sum of the standalone bars: a line terminated by M3 never reaches M2, and an M2 merge changes the request stream later seen by the owner and requester L2.

B. Removing Work Along the Path

Figure **??** explains the speedup in terms of work that no longer reaches a downstream component. Weighted across the fourteen workloads, Complete removes 22.6% of L2 tag lookups, 4.6% of unique remote reads, 48.2% of wire packets, and 30.8% of repeated wafer traversals. The different columns peak on different applications: L2 filtering is strongest for SPMV and MT, packet aggregation for AES, RELU, and KMeans, and requester- L2 reuse for MM, FIR, SC, and I2C. This separation supports the path-oriented design: no single boundary exposes all four reductions.

Fig. 12. Percentage of baseline work removed at successive boundaries. The bottom row is weighted by the corresponding baseline work count.

CuPath does not obtain these gains by increasing aggregate memory traffic. Complete reduces physical DRAM reads from 39.25 to 38.71 million (1.38%) and writes from 20.83 to 20.68 million (0.71%). The sample-weighted local L2-miss-to-DRAM-issue latency falls from 38.12 ns in Baseline to 20.34 ns over the ordinary and Filter-negative Complete issue populations. We report these populations separately in the machine-readable results; the comparison is not an assumption that every miss takes the fast path.

C. M1: Local Candidate Control

M1 is a supporting transformation rather than the dominant source of speedup. It issues 261,615 local candidates over the full suite: 252,850 reach the DRAM-issue point and 8,765 lose a race to an ordinary demand; 176,447 are eventually useful. Although these issued candidates are additional logical accesses, exact MSHR merging and demand timing leave total physical reads 0.35% lower than Baseline and increase L2 MSHR-full cycles by only 0.22%. FIR is the exception: M1 alone is 0.898× and increases physical reads and writes, so we do not claim that local prediction is universally beneficial.

The coupling experiment isolates why the Filter is still useful for mostly local workloads. Filter-only definite-miss elimination provides 1.0145× geometric mean, while ungated prediction provides only 1.0059×. Coupling the same predic-

tor to PATTERN, RESIDENT, and PENDING reaches 1.0188×. It removes 34.7% of issued candidates while retaining 96.7% of useful candidates, raising useful-per-issued accuracy from 24.0% to 35.5%. Additional DRAM reads fall from 6,206 to 5,578 and demand-delay exposure events from 145,466 to 33,066. Predictor-only records candidates but is exactly 1.0000×, confirming that observation itself is not counted as timing benefit.

Replacing Cuckoo membership with exact metadata changes the same screen only from 1.0188× to 1.0192×. The Cuckoo design observes four false positives in 273,146 queries (0.0015%). Thus the Filter does not move data or create prediction benefit by itself; it compactly rejects avoidable work and tracks the request lifecycle with little loss relative to exact metadata.

D. M2 and M3: Remote Transformation

M2 is the largest standalone contributor. Complete records 4.24 million real demands merged by exact RDMA line state and 2.52 million predicted lines piggybacked into existing owner/page packets. Because a real demand can take over a candidate before batch insertion, the aggregate counters do not expose the intersection of these two populations; we therefore do not relabel every useful prediction as an avoided packet. The exact merge and wire-packet counters independently establish the reductions.

M3 bypasses 23.41 million exact requester-L2 probes when RESIDENT and SEEN reliably prove absence, 42.6% of eligible unique-request probe opportunities. The remaining probes produce 27.23 million physical L2 hits and satisfy 28.21 million logical responses through exact waiter fanout. M3 installs 7.06 million remote-clean lines in the existing L2; 0.996 million are observed to retire unused, a 14.1% lower bound because lines still resident at termination are unclassified. Invalid ways absorb 5.35 million fills and older remote-clean lines absorb the remaining 1.71 million; no ordinary local-clean line is displaced. The largest workload-level sum of per-slice peaks occupies 37.5% of wafer L2 lines, a conservative non-simultaneous upper bound rather than an average footprint. Of the unused retirements, 160,257 carry candidate provenance and remove the associated PATTERN. A useful patterned hit instead keeps PATTERN eligible; the aggregate counter does not isolate that subset from all requester-L2 hits.

E. Sensitivity and Limitations

Figure **??** varies only the parameters needed to audit the selected point. Across four workloads, all tested points span 1.2713×–1.2817×, compared with 1.2757× for the 32K-slot, 13-bit, 64-entry, one-cycle point. Reducing the Filter to 16K slots raises insertion failures from 27 to 156,726, whereas 64K slots, 16 fingerprint bits, or a 256-entry predictor changes geometric mean by less than 0.5%. A zero-cycle lookup reaches 1.2791×, only 0.27% above the selected point; the result therefore does not depend on a free Filter lookup.

The long runs expose a limitation hidden by the short screen. Across the fourteen Complete runs, correctness-critical

Fig. 13. Four-workload sensitivity. Orange marks the selected point and blue marks alternatives; all use the same frozen binary.

RESIDENT and PENDING have zero insertion failures, but capacity pressure causes 9.42 million PATTERN and 5.81 million SEEN attempts to fail closed. These failures suppress optional prediction or reuse and cannot change demand correctness, but they bound coverage. The modest 1.187× Remote-group speedup and M1's FIR regression likewise show that CuPath is constrained by available aggregation and safe idle opportunities, not by an unlimited speculative upper bound.

A controlled simulator-runtime screen uses seven representative workloads, all five configurations, 192 drained workgroups, one host worker, and the same binary. Normalized by simulated GPU time, Complete costs 1.190× Baseline; this is simulator overhead, not hardware performance.